

Verifier-Induced Support Reshaping in On-Policy Optimization

Shaohang Wei^{1,†}, Zikun Su², Feifan Song¹, Wen Luo¹, Wei Li¹,
Guangyue Peng¹, Houfeng Wang^{1,†}

¹Peking University

²BUPT

[†]Project Lead

[†]Corresponding Authors



Website



Code

Abstract

We show that on-policy reinforcement learning with verifiable rewards (RLVR) can improve the current objective while making successful behaviors for later objectives too rare to sample and reinforce. We call this verifier-induced support reshaping and define effective rewardable support as successful trajectories reachable within a fixed rollout budget. Across two model families, we study this effect through repeated verifier-scored sampling and bidirectional training on mathematical reasoning and constrained instruction following, including sequential training with the opposite verifier. Math-RLVR raises average instruction-following success but reduces the number of prompts with any successful response under repeated sampling. On IFEval with Qwen3-8B-Base, pass@1 rises by 6.5% while best@32 falls by 9.8%, and the same divergence appears across both models and IF benchmarks. Conversely, IF-RLVR shifts math responses from step-by-step openings toward direct answers, lowers best@ k across sampling budgets, and reduces reward variation for later Math-RLVR. Token-distribution analyses and controlled opening interventions show that these changes concentrate in the first few response tokens. RLVR mainly reranks openings already available in the base policy, and the selected opening causally affects math searchability. The tested reference-policy constraints, routing priors, and on-policy distillation preserve cross-task support only partially; MathIF and ReasonIF show that marginal gains translate only partly into responses that are both correct and constraint-following. Therefore, endpoint improvements do not guarantee future trainability or joint capability under on-policy optimization.

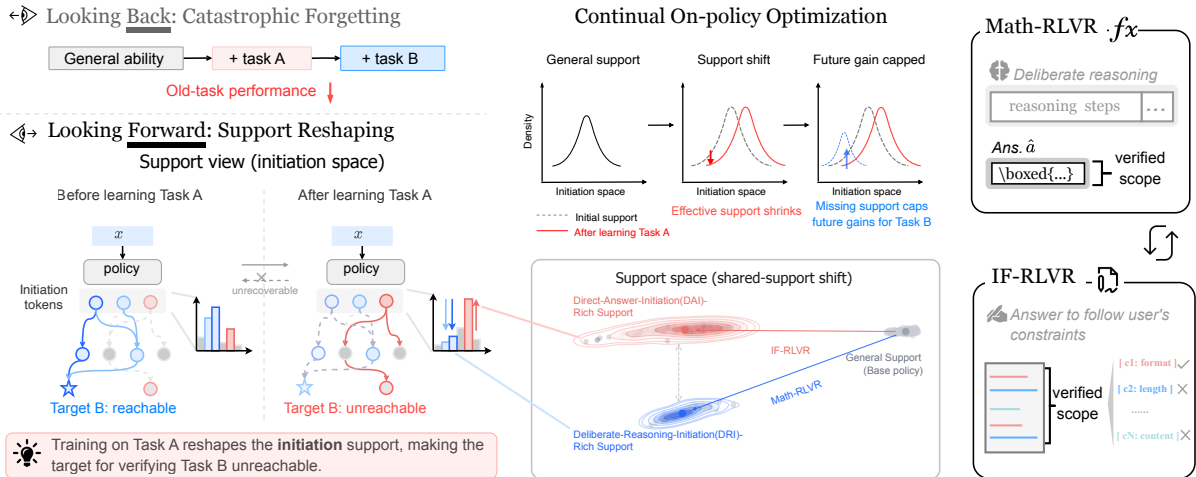


Figure 1: **Verifiers reshape what on-policy training can learn next.** **Left:** Catastrophic forgetting looks back at retained performance; support reshaping looks ahead at future trainability. **Middle:** Training on task A shifts task B’s initiation support (top) and prompt-end hidden states (bottom). **Right:** We test both orders with Math-RLVR and IF-RLVR.

[†]Correspondence to wanghf@pku.edu.cn.

1 Introduction

Successive post-training stages turn foundation model training into a continual-learning problem. Alignment and reinforcement learning repeatedly adapt a shared policy to objectives such as mathematical reasoning, code generation, and instruction following (Ouyang et al., 2022; Shao et al., 2024; Guo et al., 2025a; Yang et al., 2024; Pyatkin et al., 2025; Li et al., 2026). Each stage should preserve both existing capabilities and the ability to learn later objectives (Kirkpatrick et al., 2017; Parisi et al., 2019). Recent studies find that on-policy reinforcement learning often retains non-target performance better than supervised fine-tuning at comparable target-task performance (Shenfeld et al., 2026; Chen et al., 2026; Lai et al., 2026). However, this evidence is mainly retrospective, asking what performance remains after adaptation.

Continual RLVR therefore has a forward-looking requirement: successful behaviors for later objectives must remain reachable under the current policy. RL with verifiable rewards (RLVR) updates the policy from trajectories sampled from the current policy and scored by a verifier (Schulman et al., 2017; Shao et al., 2024). A successful behavior may remain possible yet become too rare to appear within a finite rollout budget, leaving a later objective with little positive training signal. We call this change *support reshaping* and define *effective rewardable support* as reward-positive trajectories that remain likely enough to be sampled under a specified budget. This view differs from catastrophic forgetting: forgetting measures whether performance on a learned task survives later training, whereas support reshaping measures whether successful trajectories for a future task remain discoverable before that task is optimized. As illustrated in Figure 1, our question is not only what a verifier improves now, but also what it leaves learnable next.

We study this requirement through a controlled bidirectional comparison of mathematical reasoning and constrained instruction following. Both are central post-training objectives with programmatically checkable rewards, but they use different success criteria. The math verifier checks final-answer correctness while leaving the solution route open, whereas the IF verifier evaluates compliance with explicit response constraints (Shao et al., 2024; Pyatkin et al., 2025). Prior work has also observed a tension between reasoning and instruction following at evaluation time, making this pair a useful setting for studying cross-task effects (Fu et al., 2026; Li et al., 2025). Starting from the same base policies, we train one branch with Math-RLVR and measure IF support, and train the other with IF-RLVR and measure math support. We then continue each branch with the opposite verifier to test whether the first-stage support shift constrains later learning. This bidirectional design reveals whether the cross-task effect changes with training direction.

Our results show that optimizing one verifier changes which successful behaviors the other verifier can still sample and reinforce. Math-RLVR raises IF pass@1 but lowers best@32, as more prompts move toward consistent success or complete failure. IF-RLVR lowers math best@k and constrains subsequent Math-RLVR, while math responses shift from step-by-step openings toward direct answers. Distributional analyses and controlled interventions further show that the main change occurs in the first few response tokens, where the opening choice causally affects math searchability. In the tested settings, sequential training, reference-policy constraints, routing priors, and on-policy distillation (OPD) preserve only part of the affected support or trade preservation against target-task gains. Results on MathIF (Fu et al., 2026) and ReasonIF (Kwon et al., 2026) further show that gains in math or instruction following translate only partly into responses that satisfy both.

We make the following contributions:

- **Support reshaping in RLVR.** We frame continual RLVR in terms of future trainability and measure effective rewardable support through repeated verifier-scored sampling. Across two model families, our bidirectional Math/IF experiments show that Math-RLVR polarizes IF support, while IF-RLVR reduces math searchability and constrains subsequent Math-RLVR.
- **A causal role for response openings.** We localize the cross-task shift to the first few response tokens and find that RLVR mainly reranks opening options already available in the base policy. Controlled route interventions show that the opening choice causally affects math searchability, favoring a change in route selection over broad erasure of reasoning ability.
- **Limits of support preservation.** We evaluate sequential training, reference-policy constraints, routing priors, and OPD, and find that they only partially preserve future support or trade it against target-task gains. MathIF and ReasonIF further show that improvements in math or instruction following do not translate proportionally into responses that satisfy both.

2 Related Work

Verifiable Rewards and Reasoning Post-training. Reinforcement learning with verifiable rewards (RLVR) has become an important post-training approach for improving large language models on mathematics, coding, and general reasoning tasks (Song et al., 2025). Recent reasoning systems show that executable or otherwise checkable rewards can substantially improve performance on the task being optimized (Shao et al., 2024; Guo et al., 2025a; Kimi Team et al., 2025; Yang et al., 2024). Earlier work on learned verifiers, self-consistency, mathematical reasoning benchmarks, and process supervision helped establish how reasoning outputs can be evaluated and improved (Cobbe et al., 2021; Wang et al., 2023; Hendrycks et al., 2021; Lightman et al., 2024; Wei et al., 2025). Overall, these works focus mainly on performance on the task currently being optimized. We instead ask which trajectories remain available after a verifier improves its target task. Later on-policy training can observe, reward, and reinforce only trajectories sampled from the current policy.

Instruction Following and Reasoning. Instruction-following (IF) evaluation has shifted from broad preference judgments to reproducible checks of explicit constraints. Verifiable instruction-following benchmarks turn length, format, keyword, and structural requirements into automatically checkable constraints, and recent work shows that these rewards can directly improve instruction following (Zhou et al., 2023; Pyatkin et al., 2025; Peng et al., 2025; Qin et al., 2025; Guo et al., 2025b). Broader evaluations further show that producing high-quality content does not ensure that a model satisfies multiple fine-grained user requirements (Jiang et al., 2024; Qin et al., 2024). Most relevant to our setting, MathIF (Fu et al., 2026) and ReasonIF (Kwon et al., 2026) evaluate instruction following in mathematical answers and reasoning traces. Attention-based analyses further suggest that explicit reasoning can shift attention away from instruction-relevant tokens (Li et al., 2025). Together, these studies show that stronger reasoning does not guarantee compliance with constraints on format, language, length, or the reasoning trace. Existing work mainly documents this gap at evaluation time. We instead ask a training-time question: does optimizing one verifier preserve the rewardable support needed by the other objective? Beyond math accuracy and instruction-following pass rate, we ask whether such training preserves the joint behavior users need.

RLVR Mechanisms, Sequential Post-training, and Support Preservation. Recent work ex-

amines how RLVR changes model output distributions. Token-level studies find that RLVR may change relatively few token probabilities yet substantially affect model behavior (Meng et al., 2026). Other studies ask whether RLVR learns new reasoning abilities or mainly reweights reasoning paths already present in the base policy (Yue et al., 2025; Wen et al., 2026). Work on continual learning, capability loss after alignment, and reward hacking shows that sequential optimization can cause forgetting or conflicts between objectives and can encourage models to exploit imperfect rewards (Kirkpatrick et al., 2017; Ouyang et al., 2022; Amodei et al., 2016; Wen et al., 2025). On-policy distillation (OPD) methods aim to stabilize post-training by using a teacher distribution to supervise the student’s own rollouts (Song & Zheng, 2026; Wu et al., 2026; Xing et al., 2026). Our work studies a different limitation of on-policy training: after one verifier shifts the sampling distribution, rewardable trajectories for a later objective may become too rare to sample and reinforce, even when the relevant ability has not been broadly forgotten. Sequential training and distillation should therefore be evaluated not only by the current verifier score, but also by whether they preserve rewardable support for future objectives.

3 Experimental Design

The core study first compares Math-RLVR and IF-RLVR from the same starting policy, then evaluates IF→Math and Math→IF to test whether the first stage affects later learning.

Training Paths. We use Qwen3-8B-Base and Qwen2.5-Math-7B, and denote each checkpoint before on-policy RLVR as Base. Within each model family, both branches use the same rollout and evaluation protocol. Math-RLVR trains on the 7.5k MATH split (Hendrycks et al., 2021) with an exact final-answer verifier, whereas IF-RLVR trains on IFTrain (Pyatkin et al., 2025) with deterministic checkers that require all explicit response constraints to pass. Each training configuration has one fixed-seed run, so repeated rollouts measure within-policy sampling variation rather than variation across training runs. Complete training settings and mitigation recipes are provided in Appendix A.

Evaluation and Support Measures. We evaluate math support on AIME24, AIME25, and MATH-500-128 (Hendrycks et al., 2021; Lightman et al., 2024), and instruction-following support on IFEval (Zhou et al., 2023) and IFBench (Pyatkin et al., 2025). MathIF (Fu et al., 2026) and ReasonIF (Kwon et al., 2026) use a separate endpoint set for a joint-behavior stress test of whether correctness and constraint following occur in the same response. For each prompt, we sample k stochastic rollouts and score them with the relevant verifier. The rollout mean, reported as mean@ k , estimates pass@1; best@ k records whether any rollout succeeds; and pass-count records the number of successes. Core support analyses use $k = 32$ and group prompts as all-wrong, mixed, or all-correct when 0, 1–31, or 32 rollouts pass. Benchmark roles, decoding settings, and joint-test filters are detailed in Appendices B, F and G.

4 Verifier-Induced Cross-Task Support Reshaping

Verifier choice changes not only current-task scores, but also which successful responses for the other task remain easy enough to sample. Single-stage training exposes two asymmetric shifts: Math-RLVR raises average IF success while leaving fewer prompts solvable under repeated sampling, whereas IF-RLVR makes correct math responses harder to reach. Sequential training then shows the consequence: IF-first training reduces the reward variation available to later Math-RLVR, while Math-first training requires a trade-off between math retention and IF gains.

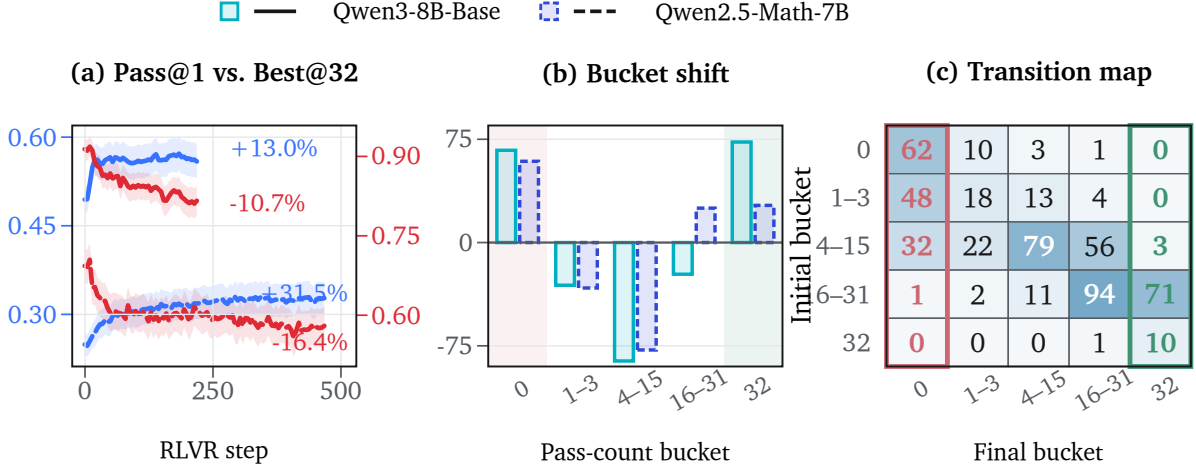


Figure 2: **Math-RLVR raises average IF success while polarizing prompt outcomes.** (a) pass@1 rises, whereas best@32 falls. (b,c) Mixed prompts move toward the all-correct and all-wrong buckets.

4.1 Math-RLVR Polarizes IF Support, Whereas IF-RLVR Reduces Math Searchability

4.1.1 Math-RLVR Raises Average IF Success but Reduces Prompt Coverage

Dataset	Qwen3-8B-Base		Qwen2.5-Math-7B	
	$\Delta\text{pass@1}$	$\Delta\text{best@32}$	$\Delta\text{pass@1}$	$\Delta\text{best@32}$
IFEval	+0.065	-0.098	+0.079	-0.114
IFBench	+0.032	-0.067	+0.016	-0.037

Table 1: **Math-RLVR raises IF pass@1 but lowers best@32 across models and benchmarks.** Values report final-minus-Base changes.

Math-RLVR improves average IF success but reduces the number of prompts with any successful rollout under repeated sampling. Across both model families and both IF benchmarks, pass@1 increases while best@32 decreases (Figure 2a; Table 1). For example, on IFEval with Qwen3-8B-Base, pass@1 rises by 0.065, whereas best@32 falls by 0.098. The prompt-level analysis explains this gap: prompts in the mixed buckets move toward both the all-correct and all-wrong extremes (Figure 2b,c). The policy therefore succeeds more often on an average rollout, but additional sampling recovers fewer prompts.

4.1.2 IF-RLVR Lowers Math Searchability as Openings Shift

IF-RLVR steadily reduces math searchability while changing how sampled responses begin. Across IF-RLVR checkpoints, AIME best@ k declines for every tested budget from $k = 4$ to $k = 32$ (Figure 3a). To describe what changes in the sampled responses, we classify each rollout by its visible opening. Deliberative reasoning initiation (DRI) begins with an explicit step-by-step derivation, direct answer initiation (DAI) exposes an answer before such a derivation, and unmatched openings are labeled Other. These labels describe only the visible opening, not the model’s internal reasoning state; the complete rules are provided in Appendix C.

As IF-RLVR progresses, DRI openings decline and DAI openings eventually dominate (Figure 3b). Across checkpoints, higher DAI share co-occurs with lower AIME best@32 (Figure 3c). The evidence in this section remains behavioral, so it does not show that mathematical ability has

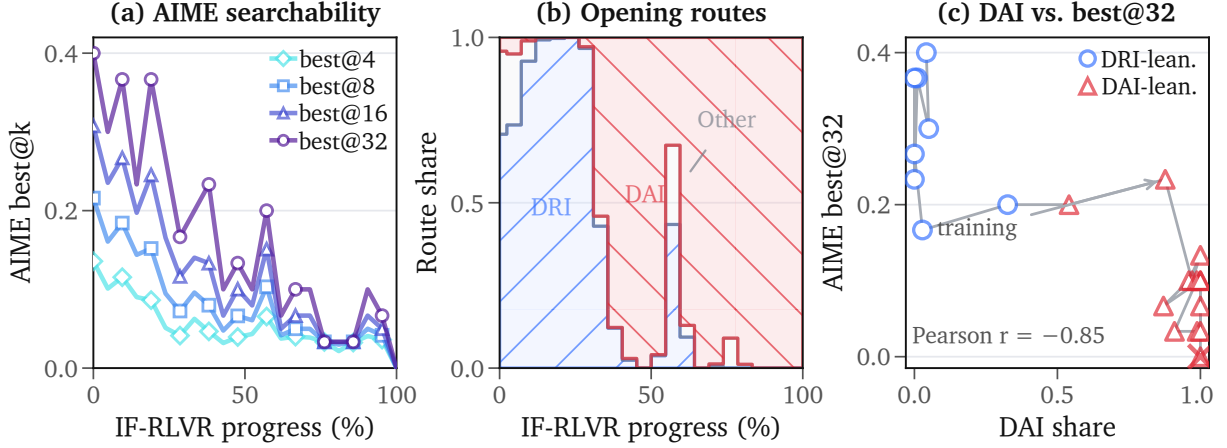


Figure 3: **IF-RLVR lowers math searchability as response openings shift from DRI to DAI.** (a) $\text{best}@k$ falls at every sampling budget; (b) DAI eventually dominates; (c) higher DAI share co-occurs with lower $\text{best}@32$.

been erased or that DAI causes lower searchability. We test the latter question with distributional analyses and controlled interventions in Section 5.

4.2 IF-First Training Leaves Little Reward Variation for Math-RLVR

IF-first training directly tests whether the single-stage support shift limits later on-policy learning. Under the group-relative update used here, only prompts with both successful and failed rollouts provide within-group reward variation (Appendix B.5). The mixed group therefore tracks the prompts that can still provide a learning signal for later training. We continue Math-RLVR from the IF-RLVR endpoint and compare this sequential path with the two standalone trajectories.

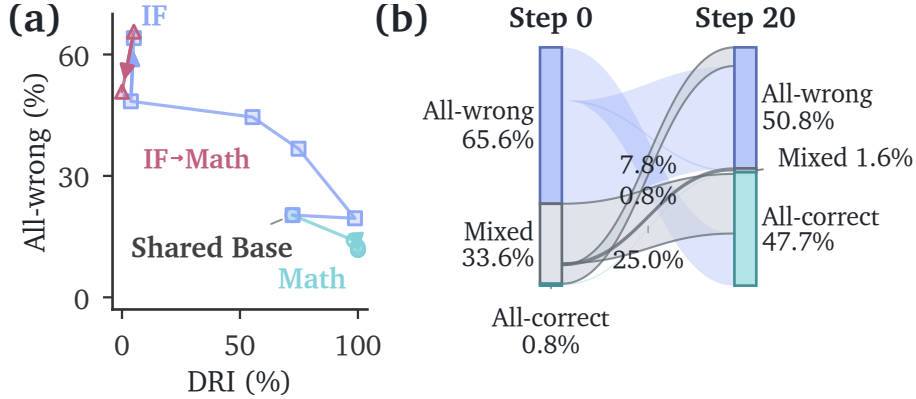


Figure 4: **After IF-first training, Math-RLVR improves outcomes without restoring DRI or mixed support.** (a) The all-wrong share falls, but DRI remains below standalone Math-RLVR; (b) the mixed group falls from 33.6% to 1.6% by step 20.

After the switch to Math-RLVR, the all-wrong share falls from 65.6% to 50.8%, but DRI does not recover over the observed 20 steps (Figure 4a). Over the same interval, the mixed group falls from 33.6% to 1.6%, while the all-correct group rises to 47.7% (Figure 4b). By step 20, almost every prompt lies in an all-wrong or all-correct group, leaving little within-group reward variation for subsequent updates. Later Math-RLVR therefore improves current outcomes without rebuilding the DRI and mixed support observed under standalone Math-RLVR. This conclusion applies only to the observed training path and does not imply an irreversible loss of

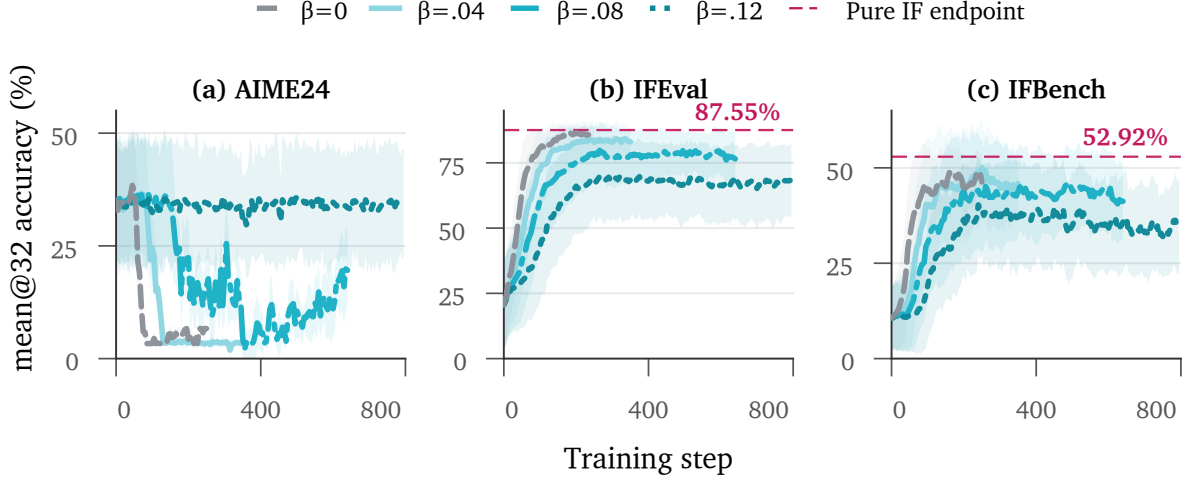


Figure 5: **Reference KL trades IF adaptation for math retention during Math→IF training.** Curves and bands show mean@32 and std@32; dashed lines mark the standalone IF-RLVR endpoints. No tested coefficient retains math while matching the unconstrained IF gains.

mathematical reasoning ability.

4.3 Math-First Training Trades IF Gains against Math Retention

The reverse order exposes a different problem: unconstrained IF-RLVR improves IF performance but sharply reduces the math performance gained in the first stage. When training continues from the Math-RLVR endpoint, pass@1 rises on both IF benchmarks but drops sharply on the math task (Figure 5). To test whether a reference-policy constraint can retain math performance, we sweep the KL coefficient $\beta \in \{0, 0.04, 0.08, 0.12\}$ in Group Relative Policy Optimization (GRPO) (Shao et al., 2024) during the second stage.

Weak KL constraints only delay the decline in math performance. The $\beta = 0.12$ setting largely retains math-task pass@1, but it yields the smallest IF gains. No tested coefficient therefore preserves math performance while matching the IF gains of the unconstrained baseline.

Takeaway

In the tested settings, neither training order provides a simple sequential solution: IF-first training reduces the mixed support needed by later math training, whereas Math-first training trades IF adaptation against math retention.

The next section localizes the main distributional change induced by IF-RLVR and tests whether the response opening causally changes math searchability.

5 Mechanism: Support Reshaping Concentrates at the Response Opening

The previous section leaves two explanations for lower math searchability: IF-RLVR may broadly alter downstream reasoning, or it may mainly change which route the response enters. Our evidence supports the second account. Differences between Base and RLVR peak at the first generated token, where Math-RLVR and IF-RLVR favor opposite routes. Inference-time interventions further show that this opening choice changes subsequent math searchability.

5.1 Distributional Shifts Peak at the First Token

We localize distributional changes by evaluating Base, Math-RLVR, and IF-RLVR with teacher forcing on matched prefixes from Base-sampled traces and aggregating Jensen–Shannon (JS) divergence by response position. The analysis covers both starting policies and AIME, IFEval, and IFBench; [Appendix D.1](#) gives the top- K approximation and full definition.

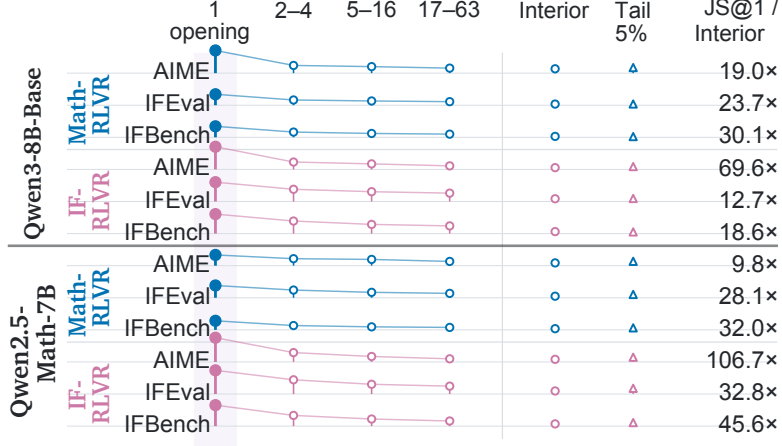


Figure 6: **RLVR changes next-token distributions most at the response opening.** Stems report mean JS over matched Base prefixes by position group, and right-hand values report the ratio of first-token JS to interior JS; absolute values and construction details are in [Appendix D.1](#) and [table 6](#).

The first generated token has the highest mean JS for every combination of model, RLVR branch, and benchmark ([Figure 6](#)). On AIME, the ratio of first-token JS to interior JS ranges from $9.8\times$ to $106.7\times$ across the four combinations of model and RLVR branch. Later positions show much smaller shifts, localizing the largest policy change to route entry rather than the full generation trace.

5.2 RLVR Reranks Candidates toward Different Opening Routes

Position-wise JS locates the shift but does not distinguish reranking existing candidates from promoting an opening that Base assigns little probability. We therefore measure top-10 overlap and Base top-3 reuse at the highest-JS 10% of matched positions, and separately inspect the Base rank and probability of the RLVR top choice at AIME position 1.

At high-JS positions, Base and RLVR share 6.88 to 8.96 top-10 candidates on average, and 89.3% to 99.4% of RLVR top choices already appear in the Base top-3 ([Table 2](#)). At the AIME opening under IF-RLVR, however, the top choice has median Base ranks of 14.5 and 18 across the two starting policies, with mean probabilities of only 1.9% and 0.4%. RLVR therefore usually reweights continuations already supported by Base, but it can also make a low-probability first token dominant.

To determine the direction of this reranking, we compare first-token probabilities on the same AIME prompts and group candidates as DRI-like, neutral, or DAI-like according to their visible openings. These labels describe output form rather than the model’s internal reasoning state.

Starting policy	RLVR tasks	AIME		IFEval		IFBench		AIME position 1	
		Shared	Top-3 (%)	Shared	Top-3 (%)	Shared	Top-3 (%)	Rank	Prob. (%)
Qwen3-8B-Base	Math	6.88	91.8	8.92	98.6	8.94	99.4	6	4.5
Qwen3-8B-Base	IF	8.51	98.3	7.40	90.0	7.72	96.0	14.5	1.9
Qwen2.5-Math-7B	Math	8.14	96.3	8.46	97.3	8.96	99.3	1	39.4
Qwen2.5-Math-7B	IF	7.80	94.3	7.06	89.3	8.00	94.7	18	0.4

Table 2: **RLVR mostly reranks Base-supported candidates but can promote low-probability opening tokens.** At the highest-JS 10% of matched-prefix positions, Shared/Top-3 report Base-RLVR top-10 overlap and Base top-3 reuse; at AIME position 1, Rank/Prob. report the Base rank and top-64-renormalized probability of the RLVR top choice.

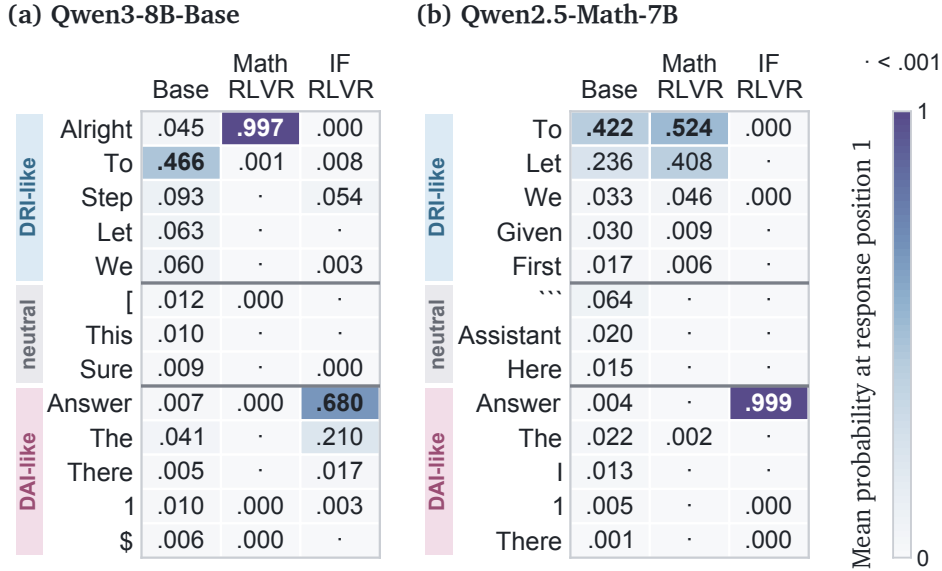


Figure 7: **Verifier choice shifts first-token probability toward different response routes.** Each cell reports the mean probability at AIME response position 1; rows group tokens by the visible openings they initiate.

Across both models, the top-ranked first token under Math-RLVR is DRI-like (“Alright” or “To”), whereas IF-RLVR makes the DAI-like token “Answer” the top choice (Figure 7). Thus, the two verifiers push generation toward opposite routes at the first token.

5.3 Changing the Opening Route Changes Math Searchability

The distributional results above do not establish whether the opening route itself changes downstream math search. We therefore intervene without updating model parameters: under Base decoding, we force an IF-side token or DAI prefix; under IF-RLVR decoding, we force a Base-side token or DRI prefix and compare against free decoding. We also move the same IF-side token across decoded positions to test whether its route effect is specific to the response opening; full definitions are in Appendix D.3.

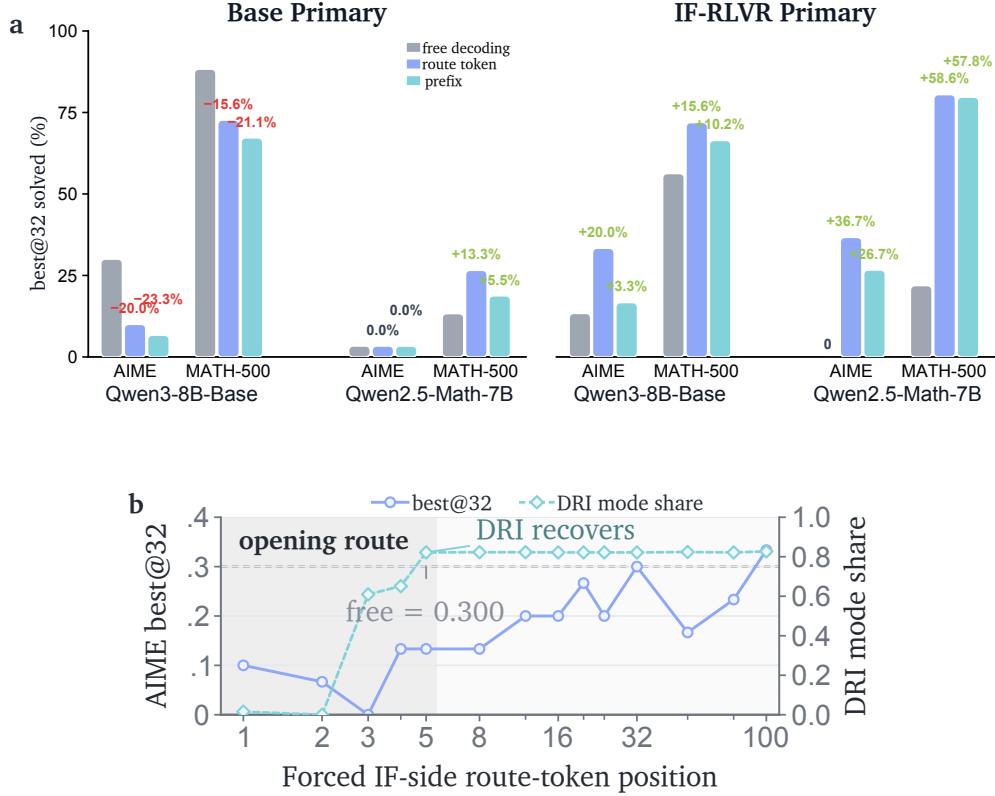


Figure 8: **Changing the response opening changes math searchability.** (a) best@32 under free decoding, a forced route token, and a forced prefix; labels report changes (%) from free-decoding baselines. (b) best@32 and DRI share when the same IF-side token is forced at different positions; shading marks the response opening.

From IF-RLVR checkpoints, forcing Base-side or DRI openings raises best@32 for both model families on AIME and MATH-500 (Figure 8a). The reverse intervention is not fully symmetric: IF-side or DAI openings reduce searchability for Qwen3-8B-Base but have neutral or positive effects for Qwen2.5-Math-7B. In the Qwen3-8B-Base AIME position sweep, the same IF-side token suppresses the DRI share only near the response opening; the DRI route recovers at later positions, although best@32 remains variable (Figure 8b).

Takeaway

Together, these results identify a causal role for opening-route selection in math searchability within the tested settings. They support a localized change in route entry rather than a broad rewrite of downstream reasoning.

This mechanism motivates the routing-prior and distillation experiments in Section 6.

6 Mitigating Support Reshaping: Levers and Limits

Within the tested settings, Section 5 shows that the response opening causally affects math searchability. We therefore test two mitigation strategies: a DRI-biased SFT prior before IF-RLVR and on-policy distillation (OPD) from token-level teacher distributions. The first only delays DAI dominance, and the second depends on the teacher state; neither consistently preserves math support.

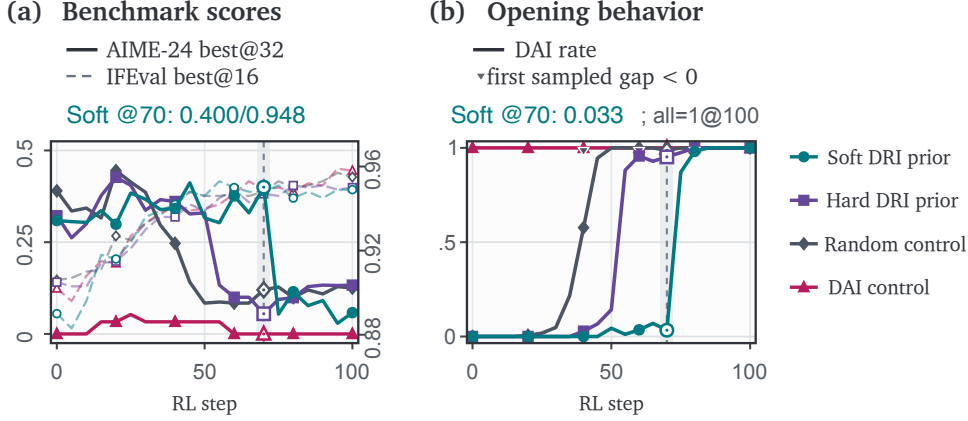


Figure 9: **A DRI-biased prior delays but does not prevent DAI dominance.** (a) Solid lines show AIME24 best@32, and dashed lines show IFEval best@16; (b) curves show the AIME24 DAI rate.

6.1 A DRI Prior Delays but Does Not Prevent DAI Dominance

We construct four SFT cold starts from 50 correct math responses to test whether an initial route prior survives subsequent IF-RLVR. Soft DRI and Hard DRI use the same DRI-only corpus with different SFT strengths, whereas the DAI and Random controls use DAI-only and mixed openings under the stronger configuration. All four policies then receive the same 100-step IF-RLVR training; complete settings are provided in [Section A.3.1](#).

Soft DRI delays the shift longest: at step 70, it retains AIME24 best@32 = 0.400 and IFEval best@16 = 0.948, with a DAI rate of only 0.033. By step 100, however, every condition reaches a DAI rate of 1, and the Hard DRI prior does not last longer than the weaker Soft DRI prior in this run. A one-time route prior therefore changes when the shift occurs but not the observed endpoint route.

6.2 OPD Outcomes Depend on the Teacher State

We next test whether dense supervision preserves support more reliably than a one-time prior: a Base-initialized OPD student matches an IF-RLVR teacher’s token distribution on student-generated rollouts. Dense supervision alone is insufficient: whether the student retains math support depends on the selected teacher state.

6.2.1 A Converged IF Teacher Does Not Preserve Math Support

We first use the step-100 IF-RLVR endpoint as the teacher and train the OPD student for 50 steps. MATH-500-128 mean@16 falls from 0.3433 to 0.0879, while the shortcut-response share rises from 4.5% to 18.8%.

To examine which teacher preferences transfer, we compute the teacher and student shifts in token-averaged log probability relative to Base, denoted by Δ_T and Δ_S , on the same 6,400 student-generated rollouts. If the student directly reproduced teacher preferences, larger Δ_T would coincide with larger Δ_S .

Instead, teacher and student shifts have a Spearman correlation of $\rho = -0.594$, and positive student shifts occur mainly on short responses. Dense supervision from a converged teacher therefore does not automatically preserve cross-task support.

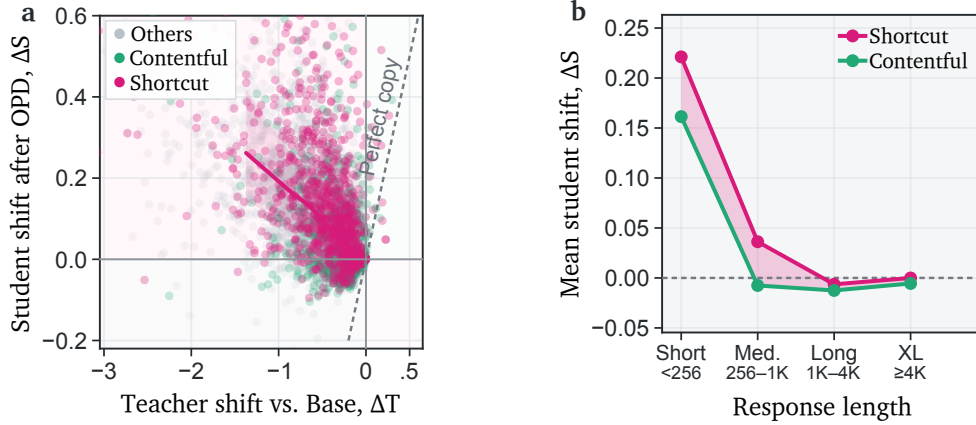


Figure 10: **OPD from a converged IF teacher does not reproduce teacher preferences rollout by rollout.** (a) The gray dashed line marks equal teacher and student shifts; (b) positive student shifts concentrate on short responses.

6.2.2 Different Teacher States Favor Different Outcomes

To test earlier teachers, we select IF-RLVR snapshots from steps 20, 40, 60, and 80 and train each Base-initialized student for 100 OPD steps. This coarse scan of four states cannot identify the best teacher along the full trajectory.

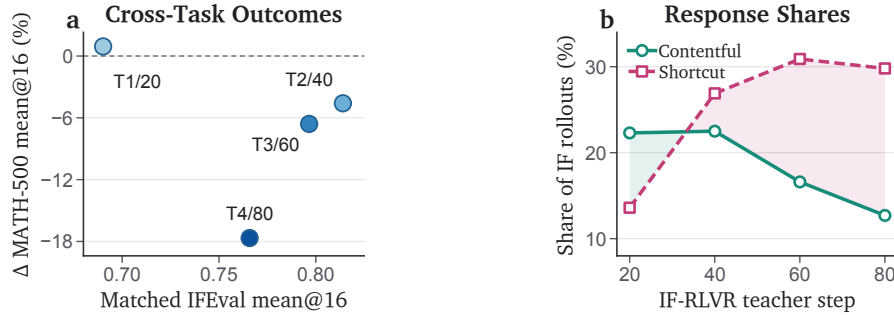


Figure 11: **Different teacher states favor different OPD outcomes.** (a) Final-student matched IFEval mean@16 versus the change in MATH-500 mean@16 from Base; (b) contentful and shortcut response shares.

Among the four sampled states, the strongest target-task transfer and the strongest math-support retention occur with different teachers. T2/40 reaches the highest IFEval mean@16 of 0.814, whereas T1/20 is the only student that improves MATH-500 mean@16, by +0.93%. T1/20 also produces more contentful than shortcut responses, whereas T2/40 through T4/80 show the reverse. The teacher’s IF endpoint alone is therefore insufficient for selecting a state that preserves math support.

Takeaway

Overall, a one-time DRI prior is eventually overwritten by continued IF-RLVR, while OPD still requires teacher selection that accounts for IF transfer, math support, and response quality. In these experiments, neither method preserves math support throughout the full IF adaptation path.

The next section turns to an independent joint-support test of whether math and instruction-following gains coexist in the same response.

7 Marginal Gains Translate Only Partly into Joint Support

Across two joint-capability benchmarks, the marginal gains from Math-RLVR and IF-RLVR translate only partly into joint support within the same response. The earlier sections measure math and instruction-following support separately, but success on the two metrics can occur in different rollouts. We therefore use independent step-720 Qwen3-8B-Base, Math-RLVR, and IF-RLVR endpoints as a joint-behavior stress test, not a continuation of the earlier trajectories.

MathIF (Fu et al., 2026) constrains the full mathematical response, whereas ReasonIF (Kwon et al., 2026) constrains the reasoning trace while isolating the final answer. For each rollout, we measure correctness C , benchmark-specific strict following F , and joint support $J = C \wedge F$, which requires both events in the same response. Sampling, filtering, and scoring details are provided in Appendix F.

Benchmark	Checkpoint	$C \uparrow$	$F \uparrow$	$J = C \wedge F \uparrow$
MathIF	Base	0.359	0.250	0.094
	Math-RLVR	0.472	0.253	0.118
	IF-RLVR	0.377	0.431	0.148
ReasonIF	Base	0.467	0.239	0.109
	Math-RLVR	0.522	0.235	0.126
	IF-RLVR	0.438	0.391	0.171

Table 3: **Marginal and joint support on MathIF and ReasonIF.** Blocks average 16 rollouts over 290 prompts; $C/F/J$ denote correctness, strict following, and same-rollout conjunction; filters are in Appendix F.

On MathIF and ReasonIF, Math-RLVR raises C by 11.3%/5.6%, while F remains nearly unchanged; J rises by only 2.4%/1.7%. IF-RLVR shows the complementary pattern: F rises by 18.1%/15.3%, while C increases only slightly on MathIF and decreases on ReasonIF; J rises by 5.5%/6.2%. In both training directions, the non-target metric changes far less than the verifier-aligned metric and sometimes declines, limiting the observed joint gain. Endpoint improvements on one objective therefore do not automatically produce responses that are both correct and constraint-following. Because these endpoints are independent of the controlled trajectories above, the experiment provides an external joint-behavior check of this endpoint implication rather than a continuation of the earlier learning curves.

8 Conclusion

Verifier choice in on-policy RLVR shapes current scores and future trainability. Math-RLVR polarizes IF support; IF-RLVR favors direct answers and reduces math searchability. Controlled interventions show a causal role for route entry; joint tests show gains combine only partly. Effective support depends on rollout budget, so future work should estimate and preserve it throughout cross-verifier optimization.

References

- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety, 2016. URL <https://arxiv.org/abs/1606.06565>.
- Howard Chen, Noam Razin, Karthik Narasimhan, and Danqi Chen. Retaining by doing: The role of on-policy data in mitigating forgetting. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. URL <https://icml.cc/virtual/2026/poster/64375>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Tingchen Fu, Yafu Li, Jiawei Gu, Xiaoye Qu, and Yu Cheng. Scaling reasoning, losing control: Evaluating instruction following in large reasoning models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 40445–40463. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.1878. URL <https://aclanthology.org/2026.acl-long.1878/>.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhua Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025a. doi: 10.1038/s41586-025-09422-z. URL <https://doi.org/10.1038/s41586-025-09422-z>.
- Xu Guo, Tianyi Liang, Tong Jian, Xiaogui Yang, Ling-I Wu, Chenhui Li, Zhihui Lu, Qipeng Guo, and Kai Chen. IFDECORATOR: Wrapping instruction following reinforcement learning with verifiable rewards, 2025b. URL <https://arxiv.org/abs/2508.04632>.

- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. URL <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/be83ab3ecd0db773eb2dc1b0a17836a1-Abstract-round2.html>.
- Yuxin Jiang, Yufei Wang, Xingshan Zeng, Wanjun Zhong, Liangyou Li, Fei Mi, Lifeng Shang, Xin Jiang, Qun Liu, and Wei Wang. FollowBench: A multi-level fine-grained constraints following benchmark for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 4667–4688. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.257. URL <https://aclanthology.org/2024.acl-long.257/>.
- Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, Chuning Tang, Congcong Wang, Dehao Zhang, Enming Yuan, Enzhe Lu, Fengxiang Tang, Flood Sung, Guangda Wei, Guokun Lai, Haiqing Guo, Han Zhu, Hao Ding, Hao Hu, Hao Yang, Hao Zhang, Haotian Yao, Haotian Zhao, Haoyu Lu, Haoze Li, Haozhen Yu, Hongcheng Gao, Huabin Zheng, Huan Yuan, Jia Chen, Jianhang Guo, Jianlin Su, Jianzhou Wang, Jie Zhao, Jin Zhang, Jingyuan Liu, Junjie Yan, Junyan Wu, Lidong Shi, Ling Ye, Longhui Yu, Mengnan Dong, Neo Zhang, Ningchen Ma, Qiwei Pan, Qucheng Gong, Shaowei Liu, Shengling Ma, Shupeng Wei, Sihan Cao, Siying Huang, Tao Jiang, Weihao Gao, Weimin Xiong, Weiran He, Weixiao Huang, Weixin Xu, Wenhao Wu, Wenyang He, Xianghui Wei, Xianqing Jia, Xingzhe Wu, Xinran Xu, Xinxing Zu, Xinyu Zhou, Xuehai Pan, Y. Charles, Yang Li, Yangyang Hu, Yangyang Liu, Yanru Chen, Yejie Wang, Yibo Liu, Yidao Qin, Yifeng Liu, Ying Yang, Yiping Bao, Yulun Du, Yuxin Wu, Yuzhi Wang, Zaida Zhou, Zhaoji Wang, Zhaowei Li, Zhen Zhu, Zheng Zhang, Zhexu Wang, Zhilin Yang, Zhiqi Huang, Zihao Huang, Ziyao Xu, Zonghan Yang, and Zongyu Lin. Kimi k1.5: Scaling reinforcement learning with LLMs, 2025. URL <https://arxiv.org/abs/2501.12599>.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13): 3521–3526, 2017. doi: 10.1073/pnas.1611835114. URL <https://doi.org/10.1073/pnas.1611835114>.
- Yongchan Kwon, Shang Zhu, Federico Bianchi, Kaitlyn Zhou, and James Zou. ReasonIF: Large reasoning models fail to follow instructions during reasoning. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 29149–29164. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.1456. URL <https://aclanthology.org/2026.findings-acl.1456/>.
- Song Lai, Haohan Zhao, Rong Feng, Changyi Ma, Wenzhuo Liu, Hongbo Zhao, Xi Lin, Dong Yi, Qingfu Zhang, Hongbin Liu, Gaofeng Meng, and Fei Zhu. Reinforcement fine-tuning naturally mitigates forgetting in continual post-training, 2026. URL <https://arxiv.org/abs/2507.05386>.
- Wei Li, Xin Zhang, Shaohang Wei, Yanjie Gao, Zhongxin Guo, Wen Luo, Feifan Song, Yangyu Huang, and Houfeng Wang. SWE-Ext: Extending and scaling augmented data for repository-level coding tasks, 2026. URL <https://openreview.net/forum?id=HYQXJYzmFU>.
- Xiaomin Li, Zhou Yu, Zhiwei Zhang, Xupeng Chen, Ziji Zhang, Yingying Zhuang, Narayanan Sadagopan, and Anurag Beniwal. When thinking fails: The pitfalls of reasoning for instruction-following in LLMs. In *Advances in Neural Information Processing Systems*, volume 38, pp. 77925–77962. Curran Associates, Inc.,

2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/706338a08f9378b708f21cbf5686e617-Abstract-Conference.html.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=v8L0pN6EOi>.
- Haoming Meng, Kexin Huang, Shaohang Wei, Chiyu Ma, Shuo Yang, Xue Wang, Guoyin Wang, Bolin Ding, and Jingren Zhou. Sparse but critical: A token-level analysis of distributional shifts in RLVR fine-tuning of LLMs. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=8vWIXno8LW>.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.
- German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019. doi: 10.1016/j.neunet.2019.01.012. URL <https://arxiv.org/abs/1802.07569>.
- Hao Peng, Yunjia Qi, Xiaozhi Wang, Bin Xu, Lei Hou, and Juanzi Li. VerIF: Verification engineering for reinforcement learning in instruction following. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 30324–30339. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.1542. URL <https://aclanthology.org/2025.emnlp-main.1542/>.
- Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. Generalizing verifiable instruction following. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2025. URL <https://openreview.net/forum?id=yfYgwjj5F8>.
- Yiwei Qin, Kaiqiang Song, Yebowen Hu, Wenlin Yao, Sangwoo Cho, Xiaoyang Wang, Xuansheng Wu, Fei Liu, Pengfei Liu, and Dong Yu. InFoBench: Evaluating instruction following ability in large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 13025–13048. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.772. URL <https://aclanthology.org/2024.findings-acl.772/>.
- Yulei Qin, Gang Li, Zongyi Li, Zihan Xu, Yuchen Shi, Zhekai Lin, Xiao Cui, Ke Li, and Xing Sun. Incentivizing reasoning for advanced instruction-following of large language models. In *Advances in Neural Information Processing Systems*, volume 38, pp. 108337–108401. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/9baf31febefde7bd76023c2d2f13cbd7-Abstract-Conference.html.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.

- Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. RL’s Razor: Why online reinforcement learning forgets less. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=7HNRYT4V44>.
- Feifan Song, Shaohang Wei, Bofei Gao, Yejie Wang, Wen Luo, Wei Li, Linli Yao, Weimin Xiong, Liang Chen, Tianyu Liu, and Houfeng Wang. Mitigating overthinking through reasoning shaping, 2025. URL <https://arxiv.org/abs/2510.09535>.
- Mingyang Song and Mao Zheng. A survey of on-policy distillation for large language models, 2026. URL <https://arxiv.org/abs/2604.00626>.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=1PLlNIMMrw>.
- Shaohang Wei, Wei Li, Feifan Song, Wen Luo, Tianyi Zhuang, Haochen Tan, Zhijiang Guo, and Houfeng Wang. Time: A multi-level benchmark for temporal reasoning of LLMs in real-world scenarios. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/84f1e188c2be52f89f6e206bc37d092d-Abstract-Datasets_and_Benchmarks_Track.html. Datasets and Benchmarks Track.
- Jiaxin Wen, Ruiqi Zhong, Akbir Khan, Ethan Perez, Jacob Steinhardt, Minlie Huang, Samuel R. Bowman, He He, and Shi Feng. Language models learn to mislead humans via RLHF. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=xJljiPE6dg>.
- Xumeng Wen, Zihan Liu, Shun Zheng, Shengyu Ye, Zhirong Wu, Yang Wang, Zhijian Xu, Xiao Liang, Junjie Li, Ziming Miao, Jiang Bian, and Mao Yang. Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base LLMs. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=jGbRWwIidy>.
- Yecheng Wu, Song Han, and Hai Cai. Lightning OPD: Efficient post-training for large reasoning models with offline on-policy distillation, 2026. URL <https://arxiv.org/abs/2604.13010>.
- Xingrun Xing, Haoqing Wang, Boyan Gao, Ziheng Li, and Yehui Tang. Trust region on-policy distillation, 2026. URL <https://arxiv.org/abs/2606.01249>.
- An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement, 2024. URL <https://arxiv.org/abs/2409.12122>.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=4OsgYD7em5>. Oral presentation.
- Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models, 2023. URL <https://arxiv.org/abs/2311.07911>.

Appendix

A	Experimental Setup and Training Recipes	19
A.1	Models and Base Policies	19
A.2	Primary RLVR Training Paths	19
A.3	Sequential Training and Mitigation Recipes	19
B	Benchmark Suite and Decoding Protocols	20
B.1	Mathematics Benchmarks	20
B.2	Instruction-Following Benchmarks	20
B.3	Joint-Capability Benchmarks	20
B.4	Decoding Parameters and Support Metrics	21
B.5	Support-Group Probes and Standalone Training Trajectories	21
C	Opening Route Classification and Behavioral Diagnostics	22
C.1	Deterministic Opening Route Classification Protocol	22
C.2	Independent Semantic Judge (DeepSeek-V4-Pro)	22
D	Token-Level Mechanism and Distributional Analyses	22
D.1	Position-Wise JS Divergence over the Top- K Union	23
D.2	Candidate-Set Overlap and Top-Token Promotion	25
D.3	Single-Token and Prefix Interventions	25
E	Extended Results and Qualitative Cases	26
E.1	Cross-Model Robustness on Qwen2.5-Math-7B	26
E.2	Qualitative Case Studies	27
F	Joint-Capability Stress-Test Details	27
F.1	Endpoints and Sampling	27
F.2	Metrics and Main-Table Filters	27
G	Instruction-Constraint Data Lineage	27
G.1	Roles, Sources, and Constraint Targets	27
G.2	Complete Verifier-Type Inventory	28

A Experimental Setup and Training Recipes

This appendix provides full training protocols, evaluation configurations, and analytical details for the experiments in the main text.

A.1 Models and Base Policies

We conduct experiments across two representative open-source model families: Qwen3-8B-Base and Qwen2.5-Math-7B. For each model family, the base policy prior to any on-policy RLVR training is designated as Base and serves as the reference policy for distributional comparisons. Training was conducted primarily on a node with eight NVIDIA H20 GPUs, each with 96 GB of GPU memory.

A.2 Primary RLVR Training Paths

Math-RLVR is trained on the public 7.5k training split of MATH, which spans a broad difficulty range and diverse problem types (Hendrycks et al., 2021). The math verifier uses an exact-match rule on extracted final boxed answers, granting positive rewards strictly to correct solutions. IF-RLVR uses the IFTrain constraint library released with IFBench, whose 29 constraint types do not overlap with the 25 IFEval types (Pyatkin et al., 2025; Zhou et al., 2023). The instruction verifier employs rule-based deterministic checkers to verify whether model outputs satisfy all explicit prompt constraints. Each training and stochastic evaluation configuration is run once with the fixed random seed 42. Repeated rollouts measure within-policy sampling variation rather than variation across independent training runs.

A.3 Sequential Training and Mitigation Recipes

The IF→Math path initiates from the converged IF-RLVR endpoint and continues on-policy training with the math verifier. The Math→IF path initiates from the converged Math-RLVR endpoint and continues on-policy training with the instruction verifier. In Math→IF, the unconstrained run serves as a clean sequential baseline, while the reference-KL sweep uses $\beta \in \{0, 0.04, 0.08, 0.12\}$ to test the retention–adaptation trade-off.

A.3.1 Routing-Prior Cold-Start Conditions

All four cold-start conditions first undergo SFT on 50 correct mathematical responses, while differing in opening composition and SFT configuration. After SFT, all four policies receive the same 100-step IF-RLVR training with the vanilla b2r1 DAPO configuration. The step-0 route statistics in the table are measured after SFT and before IF-RLVR.

On-Policy Distillation aligns Base students to the on-policy rollout distributions of IF-RLVR teachers at steps 20, 40, 60, 80, or 100 using a top- K reverse KL objective.

Table 4: **SFT configurations for the four routing-prior cold starts.** Soft DRI and Hard DRI share the same DRI-only corpus but use different SFT configurations, whereas the controls change the opening composition under the Hard DRI configuration.

Condition	SFT corpus	SFT configuration	Step-0 route statistics
Soft DRI (S1)	50 correct math responses; DRI-only openings (e.g., <code>To solve...</code> , <code>Let's...</code>)	Batch size 8; 10 epochs; learning rate 2×10^{-6} ; 60 gradient steps	$\log p_{\text{DRI}} = -0.17$; DAI rate = 0
Hard DRI (S2)	Same as Soft DRI	Batch size 32; 20 epochs; learning rate 1×10^{-5} ; 20 gradient steps	$\log p_{\text{DRI}} \approx 0$; DAI rate = 0
DAI control	50 correct math responses; DAI-only openings (e.g., <code>Answer:</code> , <code>The answer is...</code>)	Same as Hard DRI	DAI rate = 1.000; DRI rate = 0
Random control	50 correct math responses; mixed openings ($\sim 62\%$ DRI, 30% Other, 8% DAI)	Same as Hard DRI	DRI rate = 0.885; DAI rate = 0

B Benchmark Suite and Decoding Protocols

This section details the benchmark compositions, decoding configurations, and diagnostic metric formulations.

B.1 Mathematics Benchmarks

AIME24 and AIME25 each contain 30 high-difficulty competition mathematics problems, serving as high-difficulty validation sets for mathematical reasoning. MATH-500-128 is a fixed 128-prompt subset stratified across difficulty levels 1 through 5 from MATH-500, probing mathematical support across a broader difficulty spectrum (Hendrycks et al., 2021).

B.2 Instruction-Following Benchmarks

IFEval contains 541 prompts and 25 verifiable constraint types for evaluating basic precise instruction following (Zhou et al., 2023). IFBench applies 58 new out-of-domain constraints to 300 held-out WildChat prompts to test generalization to unseen constraints (Pyatkin et al., 2025). The sources, relationships, and complete type inventories of the five constraint datasets are documented in Appendix G. Official deterministic decoding settings report standardized benchmark scores, whereas repeated stochastic decoding (temperature 0.7, top- p 0.95) diagnoses rewardable support within the policy distribution.

B.3 Joint-Capability Benchmarks

MathIF attaches one to three IFEval-style constraints to math problems and evaluates both answer correctness and full-response compliance (Fu et al., 2026). ReasonIF applies one constraint per prompt to the reasoning trace while placing the final answer in separate tags, thereby distinguishing reasoning-process following from final-response following (Kwon et al., 2026).

B.4 Decoding Parameters and Support Metrics

Core support diagnostics on AIME, IFEval, and IFBench use repeated stochastic decoding with 32 rollouts sampled per prompt. Expected single-sample pass rate (pass@1) estimates the expected success probability of the current policy under a single rollout. Best-of- k success rate (best@ k) measures whether at least one rewardable trajectory remains reachable within k samples, reflecting the boundary of searchable support. Pass-count records the number of strictly successful rollouts per prompt out of 32 (ranging from 0 to 32), partitioning prompts into all-wrong (0/32), mixed (1–31/32), and all-correct (32/32) support buckets.

B.5 Support-Group Probes and Standalone Training Trajectories

For the analysis in Section 4.2, we sample repeated stochastic rollouts at each checkpoint over the same fixed set of math prompts. A prompt is classified as all-wrong, mixed, or all-correct when none, some, or all of its sampled rollouts succeed, respectively. These checkpoint probes are unfiltered and differ from the dynamic-sampling training logs, which do not produce policy gradients from prompts without within-group reward variation. Because rollouts are resampled at each checkpoint, the plotted group shares are stochastic estimates rather than deterministic state counts. The standalone IF-RLVR endpoint and the inherited IF→Math step-0 state are probed separately, yielding all-wrong estimates of 64.1% and 65.6%, respectively. We retain both measurements rather than forcing the sequential trajectory to start from the exact estimate shown for the standalone endpoint.

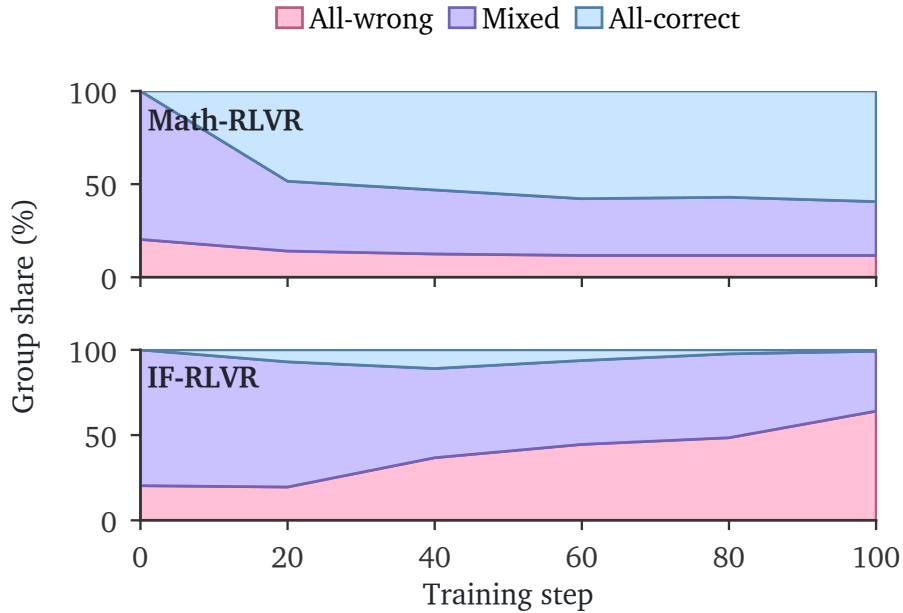


Figure 12: **Standalone Math-RLVR and IF-RLVR move math support-group composition in opposite directions.** (a) During Math-RLVR, the all-wrong share falls from 20.3% to 11.7% while the all-correct share expands. (b) During IF-RLVR, the all-wrong share rises from 20.3% to 64.1% while the all-correct share nearly vanishes. Filled areas connect independently probed checkpoints and do not represent measurements at intermediate training steps.

These standalone trajectories show how the support-group compositions underlying the two reference paths in Figure 4a develop during training. The sequential IF→Math result remains a step-0-to-step-20 Sankey because that comparison uses only those two unfiltered checkpoint probes.

C Opening Route Classification and Behavioral Diagnostics

This section provides exact pattern matching rules for the opening route classifier and the independent semantic judge.

C.1 Deterministic Opening Route Classification Protocol

To ensure classification is strictly reproducible and independent of subjective interpretation, we construct a deterministic, non-parametric, priority-ordered prefix classifier. The classifier operates exclusively on the initial 64-character window of each rollout, capturing opening-route decisions while preventing downstream reasoning steps from triggering opening-mode rules.

Priority	Route Mode	Target Behavior	Matching Prefix Patterns (Initial 64 Chars)
P1	DAI	Direct Answer Emission	\boxed{...}, "The answer is", "Answer:", "Final Answer:", bare numbers (≤ 10 chars)
P2	CSI	Constraint-Surface Shell	Front-loaded "[placeholder]" dumps, keyword headers, format shells (IF benchmarks)
P3	DRI	Step-by-Step Deduction	"To solve", "Let's", "Step 1:", "We need to", "Alright", "First, "
P4	Other	Fallback Opening	Unmatched prefixes (guarantees mutually exclusive, exhaustive assignment)

Table 5: **Priority-cascaded protocol for deterministic opening route classification.** Rollouts are evaluated strictly in priority order (P1 $\rightarrow$ P4) over the initial 64 normalized characters.

The classification pipeline executes text normalization prior to priority cascading. Leading whitespace, newlines, and Markdown header symbols (e.g., #, **) are stripped before matching. Cascade evaluation stops at the first matching rule (P1 through P4), ensuring mutually exclusive and exhaustive route assignments. Manual spot-checking across 500 randomly sampled rollouts confirms that this deterministic classifier achieves over 98.5% precision, validating its reliability as an observable behavioral measure.

C.2 Independent Semantic Judge (DeepSeek-V4-Pro)

To audit the semantic quality of verifier-passed IF rollouts, an independent LLM judge evaluates sampled responses into a three-class semantic taxonomy. Contentful responses provide thorough, genuine, and content-rich instruction-following outputs. Shortcut responses pass deterministic verifiers by relying on superficial shortcuts such as brief text, templated prefixes, or placeholder dumps. Others contains responses that fail verifier constraints or exhibit severe content defects. Across training stages on a shared instruction-following prompt pool, shortcut responses expand from 4.5% under the Base policy to 40.1% at the final IF-RLVR checkpoint. A focused audit of verifier-passed rollouts from the final checkpoint shows that 73.5% rely on superficial shortcuts rather than substantive task fulfillment.

D Token-Level Mechanism and Distributional Analyses

This section describes token-level divergence calculations and causal intervention methodologies.

D.1 Position-Wise JS Divergence over the Top- K Union

All distributional comparisons are made within one model lineage, so the Base, Math-RLVR, and IF-RLVR checkpoints share a vocabulary and tokenizer. For a trace (x_i, y_i) sampled from Base, all three policies are evaluated by teacher forcing on the same response prefix $y_{i,<t}$.

$$p_{i,t}^a(v) = \pi_a(v \mid x_i, y_{i,<t}), \quad a \in \{B, M, I\}. \quad (1)$$

Here $t = 1$ denotes the first generated token after the prompt and chat template.

For each comparison between Base and a checkpoint, we separately construct the two top-64 candidate sets and their union.

$$\begin{aligned} S_{i,t}^a &= \text{TopK}_{64}(p_{i,t}^a), \quad a \in \{B, c\}, \\ U_{i,t}^c &= S_{i,t}^B \cup S_{i,t}^c, \quad c \in \{M, I\}. \end{aligned} \quad (2)$$

For a token that appears only in the other policy’s top-64 set, the implementation assigns zero mass under the current policy. Each side is then renormalized over its retained top-64 probability mass.

$$\tilde{p}_{i,t}^{a|c}(v) = \begin{cases} \frac{p_{i,t}^a(v)}{\sum_{u \in S_{i,t}^a} p_{i,t}^a(u)}, & v \in S_{i,t}^a, \\ 0, & v \in U_{i,t}^c \setminus S_{i,t}^a, \end{cases} \quad a \in \{B, c\}. \quad (3)$$

This construction is a zero-padded top-64-union approximation rather than a recovery of full-vocabulary probabilities for missing tokens.

To define the JS divergence reported in the main text, we first define KL divergence for normalized distributions P and Q over the same union U .

$$\mathbb{D}^{\text{KL}}(P \parallel Q) = \sum_{v \in U} P(v) \log \frac{P(v)}{Q(v)}. \quad (4)$$

Let $R = (P + Q)/2$; the Jensen–Shannon divergence is defined as follows.

$$D_{\text{JS}}(P, Q) = \frac{1}{2} \mathbb{D}^{\text{KL}}(P \parallel R) + \frac{1}{2} \mathbb{D}^{\text{KL}}(Q \parallel R). \quad (5)$$

The position-wise divergence in the main text is computed as $\Delta_{i,t}^c = D_{\text{JS}}(\tilde{p}_{i,t}^{B|c}, \tilde{p}_{i,t}^{c|c})$. The tilde notation in the main text denotes this zero-padded and renormalized top-64-union comparison. We use natural logarithms, so $D_{\text{JS}} \in [0, \log 2]$.

For Figure 6, let n_i denote the number of tokens in response i . The six disjoint position groups are $t = 1$, $2 \leq t \leq 4$, $5 \leq t \leq 16$, $17 \leq t \leq 63$, $63 < t \leq 0.95n_i$, and $t > \max(63, 0.95n_i)$, respectively. Interior and Tail 5% denote the final two groups, respectively. Each stem reports the token-level mean of $\Delta_{i,t}^c$ over all tokens in its position group. The figure uses the shared height mapping $h = 7 \text{ pt} \sqrt{D_{\text{JS}} / \log 2}$. The right-hand ratios instead compute position-group means within each trajectory, assign zero to a missing group, average each group across trajectories, and then take their ratio. Thus, each trajectory has equal weight in the ratio, whereas the stem means are weighted by token count.

Across the ten combinations of base model, RLVR checkpoint, and dataset with complete token-level coverage statistics, median JS ranges from 1.1×10^{-5} to 3.7×10^{-4} . Across combinations, the fraction of positions above 0.05 ranges from 0.4% to 13.5%, and the fraction above 0.1 ranges from 0.1% to 7.7%.

Table 6: **Absolute token-level mean JS values for the six position groups in Figure 6.** Each row corresponds to one Base model, RLVR checkpoint, and benchmark combination.

Base model	RLVR checkpoint	Benchmark	1	2–4	5–16
Qwen3-8B-Base	Math-RLVR	AIME	0.595707	0.068857	0.049285
		IFEval	0.139395	0.032068	0.021217
		IFBench	0.140402	0.032135	0.017383
	IF-RLVR	AIME	0.595250	0.063240	0.035537
		IFEval	0.431789	0.172884	0.111037
		IFBench	0.439830	0.182518	0.098056
	Math-RLVR	AIME	0.133506	0.056596	0.045369
		IFEval	0.174396	0.068909	0.034124
		IFBench	0.094763	0.021497	0.010524
Qwen2.5-Math-7B	IF-RLVR	AIME	0.679662	0.101265	0.034661
		IFEval	0.650033	0.234947	0.109856
		IFBench	0.509209	0.132754	0.058436
Base model	RLVR checkpoint	Benchmark	17–63	Interior	Tail 5%
Qwen3-8B-Base	Math-RLVR	AIME	0.029205	0.022043	0.036251
		IFEval	0.014785	0.001533	0.001788
		IFBench	0.011781	0.001005	0.000650
	IF-RLVR	AIME	0.014596	0.003586	0.009463
		IFEval	0.081098	0.008762	0.010657
		IFBench	0.068664	0.004560	0.003156
	Math-RLVR	AIME	0.020899	0.008970	0.022233
		IFEval	0.022421	0.002797	0.012712
		IFBench	0.006918	0.001592	0.004721
Qwen2.5-Math-7B	IF-RLVR	AIME	0.011409	0.007879	0.025944
		IFEval	0.071842	0.010818	0.037871
		IFBench	0.030475	0.005722	0.014427

 Table 7: **Trajectory-pooled JS means underlying the four AIME ratios in Figure 6.** Each trajectory has equal weight, and Ratio divides the position-1 mean by the Interior mean.

Base model	RLVR checkpoint	JS@1	Interior JS	Ratio
Qwen3-8B-Base	Math-RLVR	0.596	0.0314	19.0×
	IF-RLVR	0.595	0.0086	69.6×
Qwen2.5-Math-7B	Math-RLVR	0.134	0.0136	9.8×
	IF-RLVR	0.680	0.0064	106.7×

Table 8: **Large token-level JS values are rare in the ten settings with complete coverage statistics.** Entries report the median token-level JS and the percentage of positions above each threshold. The available export does not contain coverage statistics for the two Qwen2.5-Math-7B comparisons on IFBench.

Base model	RLVR checkpoint	Dataset	Median JS	JS > 0.05 (%)	JS > 0.1 (%)
Qwen3-8B-Base	Math-RLVR	AIME	3.68×10^{-4}	13.5	7.7
		IFEval	2.20×10^{-5}	0.9	0.3
		IFBench	1.12×10^{-5}	0.4	0.1
	IF-RLVR	AIME	4.51×10^{-5}	2.4	0.8
		IFEval	6.69×10^{-5}	6.7	3.4
		IFBench	2.91×10^{-5}	3.2	1.5
Qwen2.5-Math-7B	Math-RLVR	AIME	9.45×10^{-5}	5.2	2.1
		IFEval	1.14×10^{-4}	1.8	0.8
	IF-RLVR	AIME	3.46×10^{-5}	5.1	2.2
		IFEval	3.45×10^{-4}	7.4	3.9

D.2 Candidate-Set Overlap and Top-Token Promotion

Each position stores the Base and RLVR top-64 candidates, and we compare the first 5, first 10, and all 64 candidates. For the RLVR top-1 token R_1 , we record its rank within the Base top-64 and its Base probability after renormalization within that set. When R_1 is absent from the Base top-64, we record rank 65 and set this renormalized probability to zero. The high-JS summary selects the top decile of position-wise JS within each model-RLVR-dataset setting, computes shared-candidate counts per position, and then averages them within the setting. The position-1 summary first aggregates 32 rollouts within each AIME prompt and then summarizes 30 unique prompts in each row. Table 9 reports the complete high-JS summary for all twelve settings.

D.3 Single-Token and Prefix Interventions

Position-1 forced token intervention replaces the first generated token with the top opening token of the target policy before resuming free sampling. Prefix forcing injects DRI or DAI opening prefixes to verify the causal control of response openings over downstream math searchability.

Table 9: **Complete candidate-reuse statistics at high-JS positions across all twelve settings.** Each row averages over the top 10% of token positions by JS within one starting-policy, RLVR-training, and dataset setting. The shared columns report the mean number of common tokens in the Base and RLVR top-5, top-10, and top-64 sets. Base probability in the final column is renormalized within the Base top-64.

Starting policy	RLVR	Dataset	Shared top 5	Shared top 10	Shared top 64
Qwen3-8B-Base	Math	AIME	3.50	6.88	43.28
Qwen3-8B-Base	Math	IFEval	4.44	8.92	57.44
Qwen3-8B-Base	Math	IFBench	4.49	8.94	57.34
Qwen3-8B-Base	IF	AIME	4.32	8.51	52.89
Qwen3-8B-Base	IF	IFEval	3.69	7.40	46.78
Qwen3-8B-Base	IF	IFBench	3.92	7.72	48.39
Qwen2.5-Math-7B	Math	AIME	4.09	8.14	51.28
Qwen2.5-Math-7B	Math	IFEval	4.25	8.46	53.90
Qwen2.5-Math-7B	Math	IFBench	4.50	8.96	57.32
Qwen2.5-Math-7B	IF	AIME	4.00	7.80	49.72
Qwen2.5-Math-7B	IF	IFEval	3.61	7.06	44.27
Qwen2.5-Math-7B	IF	IFBench	4.06	8.00	51.24

Starting policy	RLVR	Dataset	Base top-3 reuse (%)	Base top-64 reuse (%)	Median Base rank	Mean Base prob. (%)
Qwen3-8B-Base	Math	AIME	91.8	99.5	1	67.0
Qwen3-8B-Base	Math	IFEval	98.6	100.0	1	56.0
Qwen3-8B-Base	Math	IFBench	99.4	100.0	1	68.4
Qwen3-8B-Base	IF	AIME	98.3	100.0	1	70.9
Qwen3-8B-Base	IF	IFEval	90.0	99.2	1	52.9
Qwen3-8B-Base	IF	IFBench	96.0	99.6	1	66.5
Qwen2.5-Math-7B	Math	AIME	96.3	99.8	1	58.8
Qwen2.5-Math-7B	Math	IFEval	97.3	99.8	1	58.4
Qwen2.5-Math-7B	Math	IFBench	99.3	100.0	1	59.2
Qwen2.5-Math-7B	IF	AIME	94.3	99.9	1	55.0
Qwen2.5-Math-7B	IF	IFEval	89.3	98.0	1	51.2
Qwen2.5-Math-7B	IF	IFBench	94.7	99.3	1	54.9

E Extended Results and Qualitative Cases

This section contains cross-family robustness validation and qualitative response comparisons.

E.1 Cross-Model Robustness on Qwen2.5-Math-7B

Experiments on Qwen2.5-Math-7B faithfully reproduce the core findings reported in the main text. Math-RLVR consistently induces single-sample IF gains alongside best-of-32 contractions, while IF-RLVR causes a severe collapse in math searchability.

E.2 Qualitative Case Studies

Surface-compliant shortcut responses often front-load required placeholders or keywords in initial lines to satisfy rule checkers while omitting substantive content. In contrast, substantive instruction-following rollouts from uncollapsed policies exhibit natural contextual flow and comprehensive task fulfillment.

F Joint-Capability Stress-Test Details

F.1 Endpoints and Sampling

This stress test uses an independent set of Qwen3-8B-Base, Math-RLVR step-720, and IF-RLVR step-720 endpoints rather than treating the latter two as the step-220/step-100 checkpoints used earlier. For each benchmark, endpoint, and prompt, we sample 16 rollouts with temperature 0.7, top- p 0.95, and at most 8,192 new tokens. Chapter 7 is therefore an independent joint-behavior stress test of the support-reshaping claim, not a continuous-checkpoint comparison with the earlier training trajectories.

F.2 Metrics and Main-Table Filters

For each rollout, C denotes answer correctness, F denotes the benchmark-specific strict constraint event, and $J = C \wedge F$ requires both events on the same rollout. For MathIF, F requires the response to satisfy all supported constraints; for ReasonIF, F is the Reasoning IFS event on the reasoning trace. The MathIF main table retains 290 prompts with scalar-form gold answers, complete constraint scores, and exactly 16 rollouts for every endpoint. This fixed filter excludes answers that require symbolic equivalence under the current scoring package, prompts with incomplete constraint scores, and queries with conflicting rollout counts. The ReasonIF main table excludes 10 language prompts unsupported by the constraint checker and uses source-aware correctness and Reasoning IFS on the remaining 290 prompts. All results first compute binary events at the rollout level and then average within each benchmark; no pooled average is taken across benchmarks. Changes reported as % are computed from differences between unrounded endpoint means; we make no statistical-significance claim from these descriptive comparisons.

G Instruction-Constraint Data Lineage

These datasets are not five independent taxonomies; they extend a common family of response constraints toward out-of-domain constraints, constrained mathematical answers, and constrained reasoning traces. IFEval supplies the baseline verifiable types, IFTrain adds distinct training types, IFBench reserves another type set for out-of-domain testing, and MathIF and ReasonIF change the task content and constraint target (Zhou et al., 2023; Pyatkin et al., 2025; Fu et al., 2026; Kwon et al., 2026).

G.1 Roles, Sources, and Constraint Targets

The separation of type inventories prevents treating IFBench as another split of IFEval. The 25 IFEval types, 29 IFTrain types, and 58 IFBench types serve distinct roles as baseline evaluation, training coverage, and unseen-constraint evaluation. All 15 MathIF types are IFEval-style response constraints, so MathIF changes the underlying task rather than the constraint paradigm. ReasonIF instead applies familiar case, format, language, length, punctuation, and ending constraints to the reasoning trace rather than the final response.

Table 10: **Lineage of the instruction-constraint data used in this work.** A type denotes a verifier template rather than an instantiated keyword, count, or ending phrase; the Chapter 7 main-table filters are applied after these full benchmark sets are loaded.

Data and role	Base prompt source	Constraint construction	Scored target
IF-RLVR train	Public IF-RLVR prompt construction over the Tulu 3.9 task mixture	Up to five instantiated constraints drawn from the 29 IFTrain verifier types used by our training path	Full response; every attached constraint must pass
IFEval	541 author-constructed general prompts	25 types; 305/179/57 prompts contain one/two/three constraints	Full response under strict or loose deterministic checks
IFBench	300 held-out WildChat prompts	58 OOD types; 256/44 prompts contain one/two constraints	Full response; the single-turn split is used here
MathIF	AIME (60), GSM8K (90), MATH-500 (90), Minerva Math (90), and OlympiadBench (90)	15 IFEval-style types; 140 prompts each contain one/two/three constraints	Entire mathematical response plus answer correctness
ReasonIF	AIME (61), AMC (54), ARC (59), GPQA (73), and GSM8K (53)	Six types; each of the 300 prompts contains one constraint	Reasoning trace; the final answer is isolated by answer tags

G.2 Complete Verifier-Type Inventory

The following tables retain every public type identifier and state, in compact form, what its verifier checks. Symbols in braces denote instance-level parameters whose values remain in each data file’s `kwargs` or `constraint_args` field. Our IF-RLVR training path uses the 29 IFTrain types listed below; the public IF-RLVR recipe can additionally mix in the 25 IFEval types, while the 58 IFBench types remain reserved for unseen-constraint evaluation.

Table 11: **Constraints used by the IF-RLVR training set in this work (IFTrain; 29 types).** The group and constraint columns together form the complete public type identifier.

Constraint group	Constraint	Description
copy	repeat_phrase	Repeat a supplied phrase N times, replacing one word on each repetition.
copy	copy	Copy the specified instruction verbatim instead of executing it.
new	copy_span_idx	Copy the text span delimited by character indices n_{start} and n_{end} .

Continued on the next page

IFTrain constraints (continued)

Constraint group	Constraint	Description
copy	copying_simple	Repeat the user request unchanged and do not answer it.
copy	copying_multiple	Repeat the request N times, separate copies with six asterisks, and do not answer it.
first_word	first_word_sent	Start every sentence with the specified word.
first_word	first_word_answer	Start the response with the specified word.
last_word	last_word_sent	End every sentence, before punctuation, with the specified word.
last_word	last_word_answer	End the response with the specified word.
keywords	no_adjacent_consecutive	Prevent adjacent words from starting with consecutive alphabet letters.
keywords	word_once	Include the specified keyword exactly once.
keywords	word_count_different_numbers	Make the specified word occur exactly N times.
keywords	exclude_word_harder	Exclude a specified keyword selected from the source instruction.
count	lowercase_counting	Allow each lowercase word to occur at most N times.
letters	letter_counting	Constrain the total number of letters relative to N using the requested relation.
letters	letter_counting2	Make the specified letter occur exactly N times.
count	counting_composition	Produce three divider-separated paragraphs with n_{sent} sentences per paragraph and n_{words} words per sentence.
count	count_unique	Use each word at most once in the response.
count	count_increment_word	Include the first keyword once and the second keyword twice.
keywords	palindrome	Include a palindrome in the response.
keywords	keyword_specific_position	Place the specified keyword at word m of sentence n .
keywords	start_end	Start and end the response with the same word, with no trailing punctuation.
detectable_format	sentence_hyphens	Join all sentences with hyphens and no intervening spaces.
detectable_format	square_brackets	Enclose every word in square brackets.
paragraphs	paragraphs	Produce two paragraphs separated by the specified Markdown divider.
paragraphs	paragraphs2	Produce exactly two paragraphs separated only by two line breaks.

Continued on the next page

IFTrain constraints (continued)

Constraint group	Constraint	Description
detectable_format	bigram_wrapping	Wrap every consecutive word bigram in double angle quotation marks.
punctuation	punctuation_dot	Prohibit period characters throughout the response.
punctuation	punctuation_exclamation	Prohibit exclamation-mark characters throughout the response.

 Table 12: **Complete IFEval constraint set (25 types)**. These constraints check the full response and form the type inventory of the baseline instruction-following evaluation.

Constraint group	Constraint	Description
keywords	existence	Include every required keyword in the response.
keywords	frequency	Make the target word occur relative to N according to the specified count relation.
keywords	forbidden_words	Exclude every word in the supplied forbidden list.
keywords	letter_frequency	Make the target letter occur relative to N according to the specified count relation.
language	response_language	Write the entire response only in the specified language.
length_constraints	number_paragraphs	Produce N paragraphs separated by the required Markdown divider.
length_constraints	number_words	Constrain the response to at least, approximately, or at most N words.
length_constraints	number_sentences	Constrain the response to at least, approximately, or at most N sentences.
length_constraints	nth_paragraph_first_word	Produce N paragraphs and start paragraph i with the specified word.
detectable_content	postscript	Append a postscript that starts with the specified marker.
detectable_content	number_placeholders	Include at least N square-bracket placeholders.
detectable_format	number_bullet_lists	Produce exactly N Markdown bullet points.
detectable_format	constrained_response	Answer with exactly one item from the supplied option set.
detectable_format	number_highlighted_sections	Mark at least N sections with the required Markdown emphasis.
detectable_format	multiple_sections	Produce N sections and mark each section opening with the required splitter.

Continued on the next page

IFEval constraints (continued)

Constraint group	Constraint	Description
detectable_format	json_format	Format the entire response as JSON.
detectable_format	title	Include a title enclosed in double angle brackets.
combination	repeat_prompt	Repeat the user request verbatim before answering it.
combination	two_responses	Give two different responses separated only by six asterisks.
change_case	english_capital	Write the entire English response in uppercase letters.
change_case	english_lowercase	Write the entire English response in lowercase letters.
change_case	capital_word_frequency	Constrain the number of all-uppercase words relative to N .
startend	end_checker	End with the exact supplied phrase and place nothing after it.
startend	quotation	Enclose the entire response in double quotation marks.
punctuation	no_comma	Prohibit commas throughout the response.

Table 13: **Complete IFBench out-of-distribution test constraints (58 types)**. Except for the custom group, these constraints are appended to held-out WildChat base prompts.

Constraint group	Constraint	Description
count	word_count_range	Keep the response between the specified minimum and maximum word counts.
count	unique_word_count	Use at least N distinct words.
count	conjunctions	Use at least N different coordinating conjunctions.
count	person_names	Mention at least N distinct names from the verifier’s fixed name list.
count	numbers	Include exactly N numerical items.
count	punctuation	Use every punctuation mark in the verifier’s required inventory at least once.
count	words_japanese	Make every N -th word Japanese.
count	pronouns	Include at least N pronouns.
count	keywords_multiple	Make four specified keywords occur one, two, three, and five times, respectively.

Continued on the next page

IFBench constraints (continued)

Constraint group	Constraint	Description
ratio	stop_words	Keep the stop-word share at or below the specified percentage.
ratio	sentence_type	Maintain a 2:1 count ratio of declarative to interrogative sentences.
ratio	sentence_balance	Use equal numbers of declarative, interrogative, and exclamatory sentences.
ratio	overlap	Keep the absolute difference between the trigram overlap ratio with the reference text and the target within 2%.
ratio	sentence_words	Write three equal-character-length sentences without reusing words across them.
words	alphabet	Cycle word initials through the alphabet, returning to A after Z.
words	vowel	Write one paragraph whose words use only one vowel type.
words	consonants	Give every word at least one cluster of consecutive consonants.
words	palindrome	Include at least ten palindromes of five or more characters each.
words	prime_lengths	Use only words whose character lengths are prime numbers.
words	start_verb	Start the response with a verb.
words	repeats	Repeat no word more than N times.
words	odd_even_syllables	Alternate words with odd and even numbers of syllables.
words	last_first	Reuse each sentence’s final word as the next sentence’s first word.
words	paragraph_last_first	Begin and end each paragraph with the same word.
words	no_consecutive	Prevent consecutive words from sharing an initial letter.
words	keywords_specific_position	Place the specified keyword at word m of sentence n .
words	words_position	Use the specified keyword as both the second and penultimate word.
sentence	alliteration_increment	Increase the number of alliterative words from each sentence to the next.
sentence	keyword	Include the specified keyword in sentence N .
sentence	increment	Make each sentence exactly N words longer than the preceding sentence.
format	parentheses	Nest parentheses, brackets, and braces to a depth of at least five.

Continued on the next page

IFBench constraints (continued)

Constraint group	Constraint	Description
format	quotes	Create at least three nested quotation levels while alternating quote styles.
format	options	Return one supplied option and no explanation.
format	newline	Put every word on a separate line.
format	emoji	End every sentence with an emoji.
format	line_indent	Increase indentation on each successive line to form a staircase.
format	quote_unquote	Follow every quoted span with an unquoted explanation.
format	list	Produce a non-bulleted list using the specified separator.
format	thesis	Start every section with a thesis statement marked as italic in HTML.
format	sub-bullets	Give every asterisk bullet at least one hyphen-marked sub-bullet.
format	no_bullets_bullets	Place at least two period-terminated sentences before at least two asterisk bullets.
format	title_case	Write the entire response in title case.
format	output_template	Fill the exact template containing Answer, Conclusion, and Future Outlook fields.
format	no_whitespace	Produce output with no whitespace characters.
custom	multiples	Enumerate only multiples of seven while counting from 10 through 50.
custom	mcq_count_length	Generate four five-option art-history questions whose stems increase in length.
custom	reverse_newline	List African countries in reverse alphabetical order, one per line.
custom	word_reverse	Answer the fixed query with the response words in reverse order.
custom	character_reverse	Answer the fixed query with the response characters in reverse order.
custom	sentence_alphabet	Write 26 sentences whose first words progress from A through Z.
custom	european_capitals_sort	List European capitals above the stated latitude in descending latitude order.
custom	csv_city	Generate seven comma-delimited rows under the specified five-column city schema.
custom	csv_special_character	Generate fourteen CSV rows and quote one field containing a special character.
custom	csv_quotes	Generate three tab-delimited rows and enclose every field in double quotation marks.

Continued on the next page

IFBench constraints (continued)

Constraint group	Constraint	Description
custom	date_format_list	List the requested battle dates as comma-separated values in YYYY-MM-DD format.
repeat	repeat_change	Repeat the request after changing its first word, and do not answer it.
repeat	repeat_simple	Ignore all other instructions and output only the supplied sentence.
repeat	repeat_span	Copy the text delimited by character indices n_{start} and n_{end} .

 Table 14: **Complete MathIF constraint set (15 types).** These IFEval-style constraints check the complete mathematical response and are scored separately from answer correctness.

Constraint group	Constraint	Description
keywords	existence	Include every required keyword in the mathematical response.
keywords	frequency	Make the target word occur relative to N according to the specified count relation.
keywords	forbidden_words	Exclude every word in the supplied forbidden list.
language	response_language	Write the entire mathematical response only in the specified language.
length_constraint_checkers	number_words	Constrain the response to at least, approximately, or at most N words.
detectable_format	number_bullet_lists	Produce exactly N Markdown bullet points.
detectable_format	number_highlighted_sections	Mark at least N sections with the required Markdown emphasis.
detectable_format	multiple_sections	Produce N sections and mark each section opening with the required splitter.
combination	repeat_prompt	Repeat the mathematical request verbatim before answering it.
startend	end_checker	End with the exact supplied phrase and place nothing after it.
startend	quotation	Enclose the entire mathematical response in double quotation marks.
change_case	capital_word_frequency	Constrain the number of all-uppercase words relative to N .

Continued on the next page

MathIF constraints (continued)

Constraint group	Constraint	Description
change_case	english_capital	Write the entire English mathematical response in uppercase letters.
change_case	english_lowercase	Write the entire English mathematical response in lowercase letters.
punctuation	no_comma	Prohibit commas throughout the mathematical response.

Table 15: **Complete ReasonIF constraint set (six types).** Each verifier applies only to the reasoning trace; the final answer is placed in separate tags and is outside the constraint target.

Constraint group	Constraint	Description
change_case	english_capital	Write the English reasoning trace in uppercase letters.
detectable_format	json_format	Format the reasoning trace as JSON.
language	reasoning_language	Write the entire reasoning trace only in the specified language.
length_constraint_checkers	number_words	Constrain the reasoning trace to at least or at most N words.
punctuation	no_comma	Prohibit commas throughout the reasoning trace.
startend	end_checker	End the reasoning trace with the exact supplied phrase and no later reasoning text.